



Project definition

1 Introduction

The goal of the project is to develop a cloud native application that offers a set of services that provide relevant information extracted from a dataset, or multiple datasets, as a set of business capabilities and use cases. The services that are the frontend for the use cases will be provided through a REST API. Each group will choose its dataset(s) and set of services. Interconnected use cases should be emphasized, because the idea is to create a single application and not just a cluster of small independent applications. A suitable architecture should be designed. A typical architecture is organized in layers. A 3-Tier architecture provides a first layer of microservices that support the external REST API, the second layer implements the business logic, and a third layer handles the database(s) access. Communication between the services should preferably use gRPC. Any other contribution in the context of cloud computing will be valued. The main areas of contribution are microservice patterns, serverless, devops, security and data science. The resulting application is to be deployed in a cloud environment and make use of cloud services. Automation should be emphasized as much as possible for building, deploying and running the application.

2 Group organization

Each group should work as a new startup delivering business value in the form of a cloud native application. The ideal group size is between 3 and 4 students. Each participant is expected to make clear contributions that are relevant in the context of the course.

A suitable approach is for each student to participate in all phases of the project and be responsible for his/her own (small) set of microservices from its conception up to its deployment in the cloud.

3 Project development and evaluation

This project has two main deliveries, where the project is evaluated, and the individual contributions are scored. To support those deliveries, there are ten submission phases that correspond to project milestones and, again, the individual contributions are scored.

Completing these milestones in a timely manner awards each student with a score for the continuous evaluation that reflects its contribution. Completing the first delivery in a timely manner with a project release that can be deployed and run in the cloud awards each student with a score for the periodic evaluation that reflects its contribution.

The purpose of the milestones (continuous evaluation) is to set a pace for project development. A student that keeps the weekly, or bi-weekly, pace is awarded with a score. If, on the other hand, a student can't keep up with the pace (doesn't get a score for continuous evaluation) but is able to complete the two main deliveries in a timely manner it is still eligible to be awarded with full marks for the periodic evaluation. Lastly, if a student misses all deadlines will not have a chance to be awarded with a score.

The deadlines for the phases are already published at Moodle and should be considered as final. The deadline for phase 5 should be seen as the deadline for the first delivery and the deadline for phase 8 should be seen as the deadline for the second delivery.

Scores for the periodic evaluation depend on submissions to phases 5, 8, 9 and 10. These account respectively for first and second deliveries, CloudyDay and documentation/report.

Complementary information regarding the evaluation can be found at the course synopsis and slides about the course organization.

3.1 Repository

The group should use a single code repository, such as GitHub, GitLab or BitBucket, and possibly use a branch for each member to develop and test his/her contributions.

There will be two mandatory releases, one per delivery, that must be created from the main branch. The releases should be named, respectively, “Project1” and “Project2”.

The repository should add the lecturer of the course (mcalha@ciencias.ulisboa.pt) as a viewer member.

The repository should have all the source code, scripts and documentation (in a text format or Markdown format). Never store credentials, secrets, specific configurations such as IPs, URLs, binaries, images and other files that can be produced from sources. In short, minimize the attack surface through revealing the least possible and minimize the repository size, while at the same time providing all that’s necessary for the team to make progress in the development and release of the project. There are many guidelines for best practices such as <https://docs.github.com/en/enterprise-cloud@latest/admin/managing-accounts-and-repositories/managing-organizations-in-your-enterprise/best-practices-for-structuring-organizations-in-your-enterprise> and <https://docs.github.com/en/enterprise-cloud@latest/code-security/getting-started/quickstart-for-securing-your-repository>.

3.2 First delivery (Project 1)

There are 5 milestones, or phases, to support the first delivery.

3.2.1 Phase 1 – Dataset, business capabilities and use cases

In the first phase, each group must select a dataset, or multiple datasets, that will become the data source of the system to be developed.

A good dataset should have, at least, the following characteristics:

- ascii/utf8 file, stored on plain text files (like CSV or other text file format);
- size, between 1GB (or close to) and a couple of GBs;
- recent data, for example, having been produced or updated in the last couple of years;
- have interesting information suitable to be analyzed, or data mined;
- not too simple, the data set can be comprised of several files, from different sources that can be correlated.

The dataset can have information that is relevant to some scientific project or to the members of the group. The topic of the dataset is each group’s choice. A group can combine interests (and datasets).

For your reference, trendy topics are LLMs and AI agents such as <https://github.com/karpathy/llama2.c> and <https://github.com/exo-explore/llama98.c>. If you like this field make sure that the computational requirements are low, otherwise the project will not be able to be deployed without significant costs.

Examples of sources, aggregators, and lists of public datasets:

<https://www.kaggle.com/datasets>

<https://elitedatascience.com/datasets>

<https://cloud.google.com/public-datasets>

<https://ec.europa.eu/eurostat/data/database>

<https://knoema.com/atlas/Portugal/datasets>

<https://github.com/awesomedata/awesome-public-datasets>

<https://www.altexsoft.com/blog/datascience/best-public-machine-learning-datasets/>

Business capabilities are related to the main sources of revenue for a business. For instance, a car company that sells and rents cars has these two business capabilities. The use cases define the general scenarios that should be offered, like configuring a car for buying and returning a rented car.

In this phase, each student should fill in the self-assessment form and submit a Markdown file with the identification of the datasets (namely topic, size and date of release), business capabilities to be implemented and use cases that are your contributions. The document is basically descriptive, so diagrams are not necessary for this submission. Nevertheless, diagrams can also be done by following instructions at <https://docs.github.com/en/get-started/writing-on-github/working-with-advanced-formatting/creating-diagrams>. This markdown should also be added to the project repository so that it begins the README.md that documents the whole project.

3.2.2 Phase 2 – API specification

The goal for this phase is to specify a REST API using the OpenAPI format to support the use cases. The specification should cover all endpoints, parameters, responses and descriptions. The API specification should be delivered as a YAML (or JSON).

The specification can be done by using <https://editor.swagger.io/> or <https://editor-next.swagger.io/>. You can find the documentation at <https://swagger.io/docs/specification/about/>

An advantage of specifying the API in this way is that the code can be automatically generated. In this phase, each group must submit the actual specification of the REST API for the basic services. As a reminder, the application is not expected to have a client-side front-end.

In this phase, each student should fill in the self-assessment form and submit the OpenAPI specification file.

3.2.3 Phase 3 – Functional requirements and application architecture

In this phase the functional requirements and application architecture should be defined.

The functional requirements describe what the product does and focus on the user requirements. They provide a more detailed description of each use case. Functional requirements focus on the system's functionality and behavior, while use cases focus on the user's goals and needs. Functional requirements cover the entire system and all its features, while use cases cover only a subset of the system and its functions.

The specification of the architecture has two aspects: application architecture and technical architecture. The definition of an **application architecture** involves defining:

- The components like services and databases developed in the context of the project
- The interactions between these components (including the protocols such as HTTP, REST and gRPC)

This architecture is driven by the functional requirements and should be presented with a simple diagram that shows both the components, their connections and properties. The type of connections should be defined (e.g. REST, gRPC). This diagram should be accompanied by a description.

On the other hand, the **technical architecture** is driven by the non-functional requirements. This architecture mainly shows the cloud provider services/tools and any tool/script developed by the group that supports the fulfillment of the non-functional requirements. This phase requires the submission of **only the application architecture**.

In this phase, each student should fill in the self-assessment form and submit a Markdown file with the functional requirements, description of the architecture and the diagram of the application architecture.

3.2.4 Phase 4 – Implementation

The goals for this phase are:

- implement the application;
- containerize the microservices;
- the dataset(s) can be read directly from the filesystem or from a containerized database server, in this case, have the database files on the local file system (to keep the container as small as possible) and mount the local folder automatically when running the containerized database server;

It is expected that each group:

- **submits a zip (or tar.gz) file** with the project files for the current phase:
 - source files, scripts, configurations or brief instructions (don't submit the datasets, support libraries, executables or other files that can be fetched from some repository);
- **create a release called "Phase4"**;
- the repository should have basic instructions for the building and deployment of this release;
- test the building, the deployment and usage of the application.

3.2.5 Phase 5 – Deployment

The main goal for this next delivery is the **deployment of the project at the cloud**. More specifically, the goals are:

- deploy the containers to a kubernetes cluster on a cluster in the cloud (GKE)
- use a separate namespace for the pods of your project
- use an HTTP(s) ingress or Kubernetes Gateway to connect each frontend microservice to the cloud load balancer
- configure kubernetes policy for scalability (HPA and if required VPA and Cluster)
- expose only the services that really need to be accessed from the outside
- configure resource utilization through requests and limits (try to be as cost-effective as possible)
- setup autoscaling through HPA
- setup your own probes for liveness, readiness, and start-up
- implement rolling updates and rollback
- for cloud databases, consider the warehouse solution BigQuery due to the cost
- for databases deployed to kubernetes, make sure that the actual database files with your datasets are located in a Kubernetes volume, for DBs hosted at GCP (such as BigQuery) you might need a proxy installed at Kubernetes to simplify the connection, you can also use an external DB such as Mongo Atlas (but first check the restrictions that may apply for a free account). Examples of GCP hosted databases (potentially costly) include:

Cloud SQL	Cloud Spanner	Cloud Firestore	BigQuery	Cloud Bigtable
Relational databases services used for Web framework	Horizontally scalable, high-performing relational database	Document database used for mobile/web applications	Data Warehouse	Heavy reads/writes requirements, NoSQL database
Suitable for ERP, eCommerce applications	AdTech, Fintech, and so on, Global supply chain	eCommerce, Game state, IOT	Analytics	AdTech, Fintech and so on
Structured, Supports SQL	Structured, Supports SQL	Structured, No SQL, Key-value structure	Structured, Supports Sql like queries	Structured, NoSQL
Terabytes	Petabytes	No constraints	Petabytes	Petabytes
99.95% availability	99.999% availability	99.999% availability	99.99% availability	99.999% availability

It is expected that each group:

- **submits a zip (or tar.gz) file** with the project files for the current phase:
 - scripts, configurations or brief instructions (don't submit the datasets, support libraries, executables or other files that can be fetched from some repository);
- **create a release called "Phase5"**;
- the repository should have basic instructions for the building and deployment of this release;
- test the building, the deployment and usage of the application.

3.2.6 Phase 6 – Non-Functional requirements and technical architecture

In this phase, students must prepare a plan for the final implementation and deployment (phase 8). Each group is expected to propose cloud related improvements to the existing project. The improvements are expected to come from non-functional aspects of a system or new functionalities with stronger data analysis. Ideally each student proposes an improvement, but when an improvement is large multiple students can work on it.

Suggested non-functional/data science improvements:

- Microservices – Patterns such as database per service, timeout, retry, circuit breaker, CQRS, saga, cache-aside, ..., service mesh, monitoring with Prometheus, tracing
- Serverless – FaaS, serverless containers, BaaS, patterns such as event-driven, pub-sub, fan-out/fan-in, strangler, saga
- Data science – Approaches such as machine learning, AI agents, LLMs, MCP either in batch or streaming are relevant in data science, but they should be architected and deployed as cloud native solutions. An architecture can be based on Spark running in a dedicated cluster, statistics, aggregation, MLlib, graphs, streaming.
- DevOps – CI/CD pipeline, service mesh, monitoring with Prometheus, tracing, automation with terraform, ansible, GitOps
- Security – Authentication/authorization, data privacy, RBAC, secret management, secure channels, firewall, service mesh

The delivery requirement for this phase is a **PDF with the following possible sections:**

- Use cases
- Requirements (functional/non-functional)
- Architecture (application/technical)
- Deployment plan

Some of these sections might not apply to every group. The actual document contents can be adapted to better describe the group plan. The document should be synthetic.

Technical architecture was introduced in section 3.2.3.

3.2.7 Phase 7 – Implementation and scripted deployment of technical architecture/big data

The goal for this phase is to implement the technical architecture and/or big data solution that was planned in the previous phase. Because there's a wide spectrum of possibilities for this phase, it's not possible to establish guidelines.

3.2.8 Phase 8 – Automation with CI/CD

This phase should have happened before phase 4, implementation, so that all commits, pushes and pull requests were validated with robust testing. But because usually the students don't have a DevOps culture, it would probably have hindered the progress in the implementation and deployment of project 1. So now, with the project stabilized, students can focus on creating a CI/CD pipeline and writing/scripting tests to validate new commits/pushes/pull requests. The tests don't need to thoroughly validate the system, but should demonstrate the application of the various test stagings on a subset of the code.

3.2.9 Phase 9 – Testing (CloudyDay)

In this phase each group will select another group's project (target) for testing. Each group will be taking the role of an external team in charge of evaluating the maturity of a project for its final deployment. After the testing, the group will provide feedback to the target group with the test results. The actual testing will take place on June 4 (CloudyDay) the last day of classes in this semester.

Each group is expected to complete two preparations beforehand:

1. Create an entry in the [database](#) at Moodle, fill up your group number, the Topic of your dataset and upload your OpenAPI specification. The deadline for this task is **June 1, Sunday**.

2. Starting June 2, select a target group, study their OpenAPI specification and prepare the stress/smoke tests (described below).
3. Have your system running in the cloud **from 16:00 to 19:00 on June 4**. When deployed (before 16:00), edit the database entry at Moodle and add the URL to your deployed system. Make sure that with this URL and the API specification it's possible to access all endpoints. This way, other groups will be able to run their tests.
4. In the same period of June 4, complete the stress/smoke tests on the project of the target group.
5. Provide feedback of both tests to the target group, in the form of a **PDF** document documenting the tests and their results, by editing and completing your entry at the Moodle's database. The deadline to submit the test results is **June 6**.

The tests are stress testing and smoke testing. The stress testing should evaluate the capacity of several endpoints the target project in terms of requests per second (RPS). The smoke testing should be applied to other important endpoints and test their functionality. While the goal of the stress testing is to find the RPS limit for a set of endpoints, the goal of the smoke test is to assess the main endpoints from the functional point of view and check the main use cases. Each student stress tests one endpoint and smoke tests another endpoint. So, if a group has 4 members, 8 endpoints in total will be tested. Any broken, or malfunctioning, links should be reported. The stress tests can be done with <https://locust.io> or a similar platform and the smoke tests can be done with <https://www.postman.com> , <https://insomnia.rest> or similar.

3.2.10 Phase 10 – Final report

The final report documents the whole project and should follow a comprehensive structure like:

- Motivation, dataset characterization, business capabilities and use cases
- API specification
- Functional and non-functional requirements
- Architecture (application and technical)
- Implementation (high level description)
- Deployment scripts
- CI/CD pipeline with a brief description
- Test and Evaluation techniques and results (to target group)
- Corrections implemented after feedback from testing
- Cost analysis (monthly cost for the project, FinOps measures)
- Conclusions

The report is expected to document the actual contributions in the various phases, discuss the options taken throughout the project, use diagrams and tables that summarize the ideas and results and point out future improvements. You should demonstrate the final resource tuning both of the pod resources and the cluster nodes (requires a standard cluster where you can choose the node type and capacities). Also you should evaluate the total monthly cost of the solution.

The report is not expected to explain concepts, detail used frameworks, methodologies ... and should not use generic images or diagrams.

The delivery requirement for this final phase is a **PDF document with the final report**.